Unable to provision new TLS certificates across 2i2c hubs

Status: Closed

Overview

Issuance of TLS certificates was broken on at least two clusters following a recent migration to nginx-ingress. This was caused by a change to the behaviour of multiple ingresses sharing the same host.

What Happened

An attempt to launch a new hub failed after the ingress failed to obtain new TLS certificates. This was seen recently on another hub, and thus an incident was declared.

Resolution

Annotating Ingress objects that need TLS to permit cert-manager to mutate the ingress via edit-in-place.

Where we got lucky

We already learned about this possible cause when testing an internal cluster.

What Went Well?

Fixing the cert-manager configuration was quick!

What Didn't Go So Well?

No comments added

Action Items

- https://github.com/2i2c-org/infrastructure/issues/8016
- https://github.com/2i2c-org/infrastructure/issues/3278

OWNER OF REVIEW PROCESS

Angus

IMPACT TIME

Mar 31 at 16:00 to Mar 31 at 17:16

DURATION

1h 15m 39s

*All times listed in this report are in Edinburgh, London.

Timeline

Mar 31, 2026

4:00 PM

Triggered by Angus through the website.
Description: Unable to provision new TLS certificates across 2i2c hubs (View Message)
INCIDENT #2060
Unable to provision new TLS certificates across 2i2c hubs

4:04 PM Engineer adds context for manually triggering an incident

Angus Hollands #jupyterhub_u_inc_2060

Yesterday, I recreated the support deployment on 2i2c-aws-us. There, I saw that TLS certificate orders were stuck pending.

I saw 301s in the nginx-ingress logs, which I suspected were http -> https upgrades.

My first query was whether this behaviour was a new default with e.g. SSL redirect. My investigation of the various documentation for nginx-ingress and ingress-nginx suggested that this is not the case.

Then, I looked deeper at the mechanics. To satisfy the ACME issuer flow, the cert-manager has to create a temporary ingress. With further investigation, I saw evidence that the "proper" ingress (e.g. the Hub ingress) was taking priority over the transient cert-manager ingress, likely leading to the 301 we see.

4:05 PM

Angus Hollands #jupyterhub_u_inc_2060

I am now seeing this in another cluster, awi-ciroh, where I've just created a new deployment (redeploying a decommissioned workshop hub) that has the need for new TLS certificates.

4:06 PM

Angus Hollands #jupyterhub_u_inc_2060

I suspect this will happen across many clusters.

I note that we recently redeployed the JS2 cluster with this new ingress, and I did _not_ see evidence of this. But I've opened an incident for now to cover at least 2i2c-aws-us and awi-ciroh.

4:43 PM Engineer explores using the master-minion ingress model

Angus Hollands #jupyterhub_u_inc_2060

I tried using cert-manager annotations for editable ingresses last night. I think that didn't quite work either. The alternative is master-minion ingresses, and given that both approaches require us to annotate the primary ingresses, I think we should use the latter in preference to mutating ingress objects.

4:53 PM Engineer reports that master-minion is more invasive than desired. Tries editable ingresses instead.

Angus Hollands #jupyterhub_u_inc_2060

OK, that was a no-go. Although this provisioned the certs properly, it looks like the minion-master ingress mechanism requires that the `paths` are set in the minion ingress. I.e., we'd need to actually partition our ingresses into two. I don't think that's very easy to do in an agnostic way, so I'll try editable ingress objects.

5:16 PM

Resolved by Angus through Slack.
INCIDENT #2060
Unable to provision new TLS certificates across 2i2c hubs